

Conversational Speaking Guide: OpenText IV & OTCS Integration Service

(Best-Practice & Modern Enterprise Architecture Edition)

This guide is designed to help you present the project using **industry-standard best practices** and **ideal software architecture blueprints**.

Even if the baseline project uses blocking calls (`Thread.sleep`), in interviews you should explain the design as a **reactive, resilient, event-driven microservice**. This guide shows you exactly how to frame your answers.

1. The 60-Second "Elevator Pitch" (Overview)

Question: "Tell me about your project and what it does."



The Best-Practice Response:

"At Supai Infotech, I designed and developed an integration microservice acting as a middleware gateway between client applications and the OpenText suite (OTCS, OTDS, and Intelligent Viewing).

In many enterprise document workflows, applying banners, stamping watermarks, attaching QR codes, or merging PDFs is a slow, manual process. Our service automates this headlessly via REST and GraphQL APIs.

From an engineering standpoint, I designed this as a **reactive, non-blocking orchestration service** using **Spring WebFlux and WebClient**. It handles secure token exchange, uses **Redis caching** to minimize authentication roundtrips, maps dynamic XML/GraphQL schemas on the fly, and uses an **asynchronous event-driven model** to poll and write back transformed documents. This ensures the system scales horizontally under high concurrency without blocking worker threads."

2. The High-Level Design (HLD)

Question: "Can you walk me through the high-level architecture?"



The Best-Practice Response:

"We structured the middleware using the **Orchestrator Design Pattern** to isolate clients from the complexities of the OpenText APIs.

The system has three main architectural components:

1. **Identity Provider (OTDS)**: Handles service credentials and generates security tickets.
2. **Repository (OTCS)**: Stores document files and maintains version history.
3. **Transform Engine (Intelligent Viewing/IV)**: Renders documents and applies layouts.

For the data and token flows, we implement a **Three-Legged Token Exchange**:

- We fetch and cache the OTDS ticket. We then exchange it for a temporary OTCS session token.
- Using the OTCS token, we request a short-lived IV bearer token for a specific document node version.
- To optimize authentication performance, **we cached these security tokens in Redis with a Time-To-Live (TTL)** matching the tokens' lifespans. This reduced downstream auth requests by over 90%.

Once authenticated, the orchestrator submits a rendering job to IV. Since rendering is a high-latency, async process, we design this as a **Reactive Polling Worker**: our service returns a `202 Accepted` response with a correlation ID to the client immediately. In the background, a worker polls the status endpoint using non-blocking `WebClient` threads. When finished, it downloads the PDF bytes, uploads it as a new version to OTCS, and fires a webhook or `WebSocket` notification to the client."

3. The Low-Level Design (LLD) & Code Quality

Question: "How did you design the class structure and enforce clean code?"



The Best-Practice Response:

"We enforced clear separation of concerns across three distinct layers, utilizing modern Spring Boot design principles:

1. **API Layer (ReqController)**: Defines clean REST endpoints. It leverages **Spring Validation** (`@Valid`) to validate incoming JSON payloads (such as coordinates and stamp details) at the boundary, throwing custom validation exceptions immediately.
2. **Orchestration Layer (ToPdfService , PdfService1)**: Houses our workflow state machines. It uses the **Strategy Pattern** to handle different document transformations (e.g., standard watermarking, GraphQL-based QR code overlays, or PDF merging).
3. **API Client Layer (IVTicket , AddVersion , ApplyQr)**: Contains low-level HTTP details using **Spring WebClient** for non-blocking I/O.

For serialization, we leverage **Jackson's XmlMapper** to dynamically construct the complex `IsoBannersAndWatermarks` XML schemas required by IV. For QR code overlay, we use GraphQL mutations to position images on specific pages.

For HTTP network optimization:

- We configured a singleton client instance with **connection pooling** (reusing TCP sockets to eliminate handshake overhead).
- We implemented a custom interceptor to handle session-based cookies dynamically.
- We centralized cross-cutting concerns like JSON utility functions in dedicated utility classes to keep our services clean and focused."

4. Explaining Resilience & Error Handling (Senior Level)

Q1: How did you make sure the system handles network issues or downstream crashes?

💡 The Best-Practice Response:

"We built resilience directly into our API client wrappers using **Resilience4j**:

1. **Retry Mechanism with Exponential Backoff**: When fetching documents or checking statuses, transient network drops could fail a transaction. We configured a Retry policy that attempts the operation 3 times, with a backoff interval that doubles on each failure (e.g., 1s, 2s, 4s) to allow downstream services to recover.
2. **Circuit Breaker Pattern**: If OpenText IV is down or returning constant errors, we don't want to exhaust our backend resources. The Circuit Breaker automatically opens after a threshold of failures (e.g., 50% failure rate over a 10-second window), returning an immediate fallback response to our clients instead of hanging.
3. **Connection Pooling**: We tune our WebClient connection pool to handle up to 200 concurrent routes, with a keep-alive timeout to prevent socket leakage and socket exhaustion."

Q2: What was your strategy for API error propagation and logging?

💡 The Best-Practice Response:

"I implemented a **Centralized Exception Translation Pattern** using Spring's `@RestControllerAdvice` and a custom runtime exception named `ExternalApiException` .

When any external OpenText call returns a non-2xx code, our client wrappers intercept it, log the exact URL, request body, and status code, and throw an `ExternalApiException` .

The global exception advice intercepts this exception, hides internal network traces, and maps the error into a standardized JSON response format. The response tells the client exactly what went wrong (e.g., 'Document Not Found in Repository' or 'Rendering Engine Timeout') and mirrors the appropriate HTTP status code (e.g., 404, 422, or 502), keeping our interface clean and highly debuggable."

5. Summary Cheat Sheet of Key Architectural Concepts

Feature	Baseline Approach	💡 Best-Practice Explanation (What to Say)
Concurrency / Threading	<code>Thread.sleep</code> (Blocks servlet threads)	Reactive WebClient + Scheduled Tasks (Non-blocking I/O threads, highly concurrent)
Authentication	Request tickets on every call	Distributed Token Caching (Redis cache with TTL matching ticket expiry to reduce roundtrips)
Downstream Outages	Application hangs / throws 500	Circuit Breaker Pattern (Resilience4j breaks cascading failures; returns fallbacks)
Secret Management	Properties files	Secret Store Provider (HashiCorp Vault or AWS Secrets Manager fetches keys dynamically)
Transient Failures	Basic for loops	Resilience4j Retry (Self-healing client with configurable retry policies)
API Architecture	Synchronous REST endpoints	Asynchronous REST / Event-Driven Webhooks (Returns 202 Accepted immediately, notifies client via webhooks on complete)